Yash Mahesh Kamerkar

(201) 589-7163 | yashkamerkar1407@gmail.com | LinkedIn | Portfolio | San Jose, CA

Professional Summary

Software Engineer and AI Engineer with 3 years of experience building production-grade **microservices** and **Large Language Model (LLM)** applications across logistics, enterprise technology, and ride-hailing. Delivered **Java** and **Spring Boot** services processing 45,000 daily purchase requests at Dell Technologies; architected **LangGraph** multi-agent orchestration and **Retrieval-Augmented Generation (RAG)** pipelines at Uber serving 12,000 compliance documents. Proficient in **Model Context Protocol (MCP)**, **pgvector**, **OpenAI API**, **Anthropic Claude API**, **XGBoost**, and cloud-native deployment on **Amazon Web Services (AWS)** and **Microsoft Azure**.

Technical Skills

Languages: Python, Java, JavaScript (ES6+), TypeScript, SQL, Bash
AI/ML & Generative AI: LangChain, LangGraph, OpenAI API, Anthropic Claude API, Amazon Bedrock, Retrieval-Augmented Generation (RAG), Large Language Models (LLMs), Prompt Engineering, XGBoost, scikit-learn, PyTorch, Model Context Protocol (MCP)
Frameworks & Libraries: FastAPI, Flask, Spring Boot, Spring MVC, Hibernate ORM, Next.js, React.js, Streamlit, Pydantic, Pandas, NumPy, JUnit 5
Databases: PostgreSQL, pgvector, MySQL, MongoDB, Pinecone, ChromaDB, Supabase, Redis, Oracle, SQLite
Cloud & DevOps: Amazon Web Services (AWS), Amazon SageMaker, Microsoft Azure, Azure OpenAI Service, Docker, Kubernetes (K8s), Jenkins, GitHub Actions, Terraform, CI/CD Pipelines, Vercel
Tools & Methodologies: gRPC, WebGPU, REST APIs, Microservices Architecture, Git, GitHub, JIRA, Postman, Linux, Agile/Scrum

Experience

Uber USA
Software Engineer | AI Engineer *Aug. 2026 – Present*

- Architected a **gRPC**-based trip-event aggregation **microservice** in **Java** and **Spring Boot**, consolidating driver telemetry from 4 upstream location feeds and cutting duplicate event processing incidents from 9 to 2 per week.
- Engineered a **RAG** pipeline using **LangChain**, **OpenAI API**, and **Pinecone** to power an internal driver policy assistant, cutting policy lookup time from 15 minutes to under 50 seconds across 12,000 compliance documents.
- Designed a multi-agent orchestration workflow in **LangGraph** and **Anthropic Claude API** to automate driver escalation triage, routing 1,200 daily support tickets across 6 specialized agent nodes and reducing manual dispatch overhead by 18 hours per week.
- Integrated **pgvector** semantic search over 200K+ driver support embeddings into the policy assistant retrieval layer, lifting mean reciprocal rank from 0.61 to 0.89 and reducing irrelevant document retrievals from 34 to 3 per day.
- Deployed **MCP** servers to expose 12 internal trip-data **REST API** endpoints as structured **LLM** tool calls, enabling 6 downstream teams to build agent-native integrations without modifying core backend services.
- Built a real-time surge-demand prediction model using **XGBoost** on **AWS**, ingesting 14 live geospatial signals and improving driver pre-positioning accuracy from 61 to 89 correct dispatch decisions per 100 surge windows.
- Launched an **Amazon Bedrock**-powered analytics assistant converting natural-language queries into SQL against the trip data warehouse, eliminating 22 weekly ad-hoc data requests and cutting analyst turnaround from 48 hours to 4 minutes.

Universal Courier India
Software Engineer | AI Engineer *Jul. 2023 – Aug. 2024*

- Integrated 3 third-party carrier **REST APIs** (UPS, FedEx, USPS) using **Python** and **FastAPI** to consolidate real-time shipment status for 8,000 daily parcels into a unified customer-facing tracking dashboard.
- Designed a delivery ETA prediction model using **scikit-learn** trained on 2 years of historical shipment records, reducing average estimation error from 3.4 hours to 1.1 hours across 5 regional delivery zones.
- Built an **LLM**-powered customer support assistant using **LangChain** and **OpenAI API** to handle inbound delivery inquiry tickets, automating resolution of 420 repetitive queries per month and cutting average first-response time from 6 hours to 12 minutes.
- Constructed **REST API** endpoints in **FastAPI** to expose real-time route and package status data to 3 internal logistics dashboards, supporting 150 concurrent dispatcher users during peak morning scheduling windows.

- Developed an **ETL** pipeline in **Python** using **Pandas** and **SQLAlchemy** that ingested and normalized 180 GB of daily shipment records from 6 warehouse management systems, enabling downstream analytics and executive reporting.
- Automated nightly route-optimization batch jobs using **Jenkins** and **Docker** on **Microsoft Azure**, compressing manual scheduling overhead from 4 hours to 35 minutes per dispatch cycle across 3 regional distribution hubs.

Dell Technologies India
Software Engineer *Sep. 2021 – Jul. 2023*

- Implemented order-management **microservices** in **Java 11** and **Spring Boot 2.x** for Dell's supply chain platform, processing 45,000 daily purchase requests across 6 interconnected service modules.
- Migrated 3 legacy **Oracle PL/SQL** batch jobs to **Python**-based **ETL** pipelines using **Pandas** and **SQLAlchemy**, reducing nightly batch processing time from 6 hours to 2.5 hours.
- Authored **JUnit 5** and **Mockito** test suites for 12 core service modules, eliminating 14 critical regression defects before each production release and adding 1,500 net tested lines across the codebase.
- Containerized **Spring Boot** services via **Docker** on **Kubernetes (K8s)** clusters, enabling zero-downtime rolling updates across 4 staging and production environments.
- Streamlined **RESTful API** contracts for 3 product catalog endpoints using **Spring MVC** and **Hibernate ORM**, cutting internal portal integration onboarding time from 5 days to 2 days.
- Optimized **PostgreSQL** query performance for 5 high-frequency reporting endpoints by introducing composite indexes and rewriting 3 N+1 query patterns, reducing average database response time from 2.1 seconds to 340 ms.
- Refactored a monolithic inventory reporting module into 2 independently deployable **Spring Boot** services, cutting module build time from 14 minutes to 5 minutes and removing 3 cross-team deployment dependencies.

Projects

Context Window | *Python, LangGraph, Claude API, MCP, FastAPI, pgvector, Next.js, GitHub Actions* Demo

- Architected an autonomous newsroom of 11 collaborating AI agents using **LangGraph** and **Anthropic Claude API** that publishes a fact-checked daily AI briefing with zero human input, directing 4 parallel reporter agents to scan 250 sources (GitHub, PyPI, arXiv, Hugging Face) via custom **Model Context Protocol (MCP)** servers.
- Developed a claim-level fact-checking engine backed by **RAG** and pgvector semantic deduplication, agreeing with human judges on 59/60 adversarial claims (deliberately false or subtly altered) by tying every published sentence to quoted source evidence.

Medical Bill Auditor | *Python, LangGraph, BM25, ChromaDB, MCP, Gemini API, Streamlit, Pydantic* Demo

- Built a **LangGraph** agentic system auditing medical bills and insurer EOBs across 6 error types, achieving 100/100 accuracy on 100 held-out synthetic bills versus 45/100 for a standalone **LLM** and 75/100 for a ReAct agent with identical tools.
- Implemented corrective **RAG** over the Medicare coding manual using hybrid BM25 plus ChromaDB vector retrieval fused via Reciprocal Rank Fusion (MRR 0.93), Gemini-based relevance grading and query rewriting, and word-for-word source-quote verification confirming 100/100 retrieved citations.
- Eliminated false positives by replacing **LLM** error decisions with deterministic rule checks enforced by **Pydantic** schemas and MCP tool-call fingerprinting (validating each agent's tool invocations against an expected call signature), lifting precision from 0.43 to 1.00 on the held-out bill set; a separate 30-case adversarial red-team suite returned 0 wrong findings with all injection attempts deflected.

Mettle | *TypeScript, React, WebGPU/WGSL, WebLLM, Python, Supabase, Playwright, GitHub Actions* Demo

- Engineered a browser-native **LLM** benchmark running SmolLM2-360M on the visitor's GPU through **WebGPU** with a custom WGSL matrix-multiply timing harness separating download, load, time-to-first-token, and generation phases within 4% of WebLLM's internal measurements.
- Ran a pre-registered evaluation on 500 frozen test questions revealing that 4-bit compression dropped instruction-following from 48/100 to 9/100 on a dedicated 100-prompt subset at only 1.12× speed gain (CI: 1.11–1.14), exposing a 19-point arithmetic accuracy loss while other task categories improved.

FlightSense | *Python, XGBoost, AWS SageMaker, Amazon Bedrock, AWS Lambda, Amazon S3, RDS, EC2* arXiv:2605.07364

- Co-developed FlightSense, an end-to-end **AWS MLOps** platform for real-time flight-delay prediction integrating an **Amazon Bedrock** assistant that answers delay queries via tool calls on live NOAA weather data through AWS Lambda and Amazon S3.
- Deployed a production **Amazon SageMaker** pipeline with daily NOAA weather ingestion, 40 hyperparameter tuning jobs, and a real-time inference endpoint; validated XGBoost-based ensemble outputs on unseen future months with bootstrap confidence intervals, matching CatBoost and LightGBM baselines while nearly doubling precision over a schedule-only baseline.

Education

Stevens Institute of Technology Hoboken, NJ
Master of Science in Computer Science *Sep. 2024 – May 2026*